An Enhanced and High-Performance Algorithm for Accurate Named Entity Recognition in Legal Text

Varsha Naik
Department of Computer Engineering and Technology
Dr. Vishwanath Karad MIT World Peace University
Pune, India
varsha.powar@mitwpu.edu.in

Aniket Rahalkar
Department of Computer Engineering and Technology
Dr. Vishwanath Karad MIT World Peace University
Pune, India
aniket.rahalkar@mitwpu.edu.in

2025 International Conference on Sustainable Energy Technologies and Computational Intelligence (SETCOM) | 979-8-3315-2054-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/SETCOM64758.2025.10932533

***Abstract*—The rise of digital technologies results in the generation of large volumes of legal data. Although natural language legal texts are easily understood by humans, they pose significant challenges for machine processing. Converting this information into a machine-readable format is essential for efficient legal data analysis. In this paper, we propose resource-efficient method for extracting and classifying named entities within legal documents. We use rule-based learning and regular expressions to identify legal entities like parties, courts, and case references. Once extracted, these entities are classified into categories such as individuals, legal organizations, and jurisdictions. To improve both accuracy and processing speed, we implement a modified K-Means algorithm clustering the extracted entities. This clustering step refines the classification by grouping entities based on their shared attributes. The proposed method significantly enhances the speed and precision of Named Entity Recognition (NER) for legal texts, enabling faster and more efficient legal data processing.**

***Index Terms*—Named Entity Recognition, Legal Text, Rule-Based Learning, K-Means Clustering, Unsupervised Learning, Legal Data Analysis**

I. Introduction

The vast amount of text data generated from various legal documents, such as court judgments, legal correspondence, or case law summaries, holds significant value in uncovering critical insights about the entities involved. Extracting this information is essential in numerous scenarios, such as legal decision-making, case analysis, or policy formulation for both individuals and organizations. The ability to efficiently process and analyse this legal data can be instrumental in shaping legal strategies and supporting informed decision-making in the legal domain. The paper proposed by Raviraj Joshi [1] presents the L3Cube-MahaNER dataset, unique paper for Named Entity Recognition in Marathi, consisting of 25,000 annually create these components, incorporating the applicable criteria that follow. Annotated sentences across eight entity classes. It benchmarks various deep learning models, including MahaBERT and mBERT, demonstrating that MahaBERT achieves the best performance, thereby contributing significantly to NER resources for low-resource languages like Marathi. Paper proposed by [2] presents the development of Marathi SentiWordNet, a Marathi sentiment lexicon, utilizing a synset projection approach from Hindi and English SentiWordNets. Evaluating the effectiveness of model called Bi-LSTM based SVM classifier for sentiment classification, demonstrating improved performance over traditional methods and addressing challenges in analyzing Marathi sentiments. Contents in the paper proposed by Shashi Prakash Tripathi [3] presents a computationally efficient approach for Named Entity Recognition (NER) that combines rule-based learning, regular expressions, and an enhanced K-Means clustering algorithm to extract and categorize named entities from text. The proposed SimNER algorithm outperforms existing NER taggers, achieving higher accuracy (95.31%) and faster processing times, making it suitable for large datasets

Named entities in legal texts are often not limited to single words; they can span multiple words, such as Supreme Court of India, Ministry of Justice, or John Doe. Therefore, accurately identifying the boundaries of these entities is essential. Once identified, the next step is to classify them. One approach involves using a pre-defined lexicon of legal terms and entities. While this approach may be effective for certain standardized terms like locations, less efficient for individual names, law firms, or organizations. Legal corpus for such entities are continually expanding, often ambiguous, and sometimes overlapping, naming organizations after people or places. An alternative approach to legal NER involves using classifiers that apply the Inside-Outside-Beginning (IOB) tagging scheme to parts of speech, allowing for more accurate identification and categorization of legal entities based on their linguistic and contextual features.

Existing Named Entity Recognition (NER) models often struggle with recognizing multi-word and ambiguous entities in legal texts due to their contextual nature and the specific linguistic patterns involved. Legal entities, such as "Supreme Court of India" or "Negotiable Instruments Act," frequently span multiple words and require domain-specific understanding, making them challenging for

979-8-3315-2054-0/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:03:29 UTC from IEEE Xplore. Restrictions apply.

general-purpose NER tools. Additionally, scalability issues in handling large legal datasets hinder practical applications, as current systems are often not equipped to manage the computational demands of such data. General-purpose NER tools lack customization for legal entities, such as court names, case numbers, and legal acts, which are unique to the field and not well-supported by traditional models. Current clustering methods, like basic K-Means, fail to achieve optimal clustering for high-dimensional legal data due to issues with irregular and overlapping entity distributions. The absence of efficient and scalable frameworks for organizing and analyzing legal data further limits real-world usability, underscoring the need for advanced solutions that can provide accurate, domain-specific entity extraction and clustering.

Consider the following text:
Author: M Kadri Anchor Capitals of India Ltd. and vs State Of Gujarat and Anr. on 22 April, 1998 The petitioners have filed this application under section 482 of the Code of Criminal Procedure, 1973 (to be referred to as "the Code"), to quash the complaint which is registered as Criminal Case No. 2793 of 1996 before the Metropolitan Magistrate Court No. 16 Ahmedabad, for the offences punishable under section 138 of the Negotiable Instruments Act, 188.
('Anchor', 'NNP'), ('Capitals', 'NNP'), ('Of', 'IN'), ('India', 'NNP'), ('Ltd.', 'NNP'), ('And', 'CC'), ('vs', 'JJ'), ('State', 'NNP'), ('Of', 'IN'), ('Gujarat', 'NNP'), ('And', 'CC'), ('Anr', 'NNP'), ('on', 'IN'), ('22', CD'), ('April', 'NNP'), ('1998', 'CD'), ('Author', 'NNP'), ('M', 'NNP'), ('Kadri', 'NNP'), ('The', 'DT'), ('petitioners', 'NNS'), ('have', 'VBP'), ('filed', 'VBN'), ('this', 'DT'), ('application', 'NN'), ('under', 'IN'), ('section', 'NN'), ('482', 'CD'), ('of', 'IN'), ('the', 'DT'), ('Code', 'NNP'), ('of', 'IN'), ('Criminal', 'NNP'), ('Procedure', 'NNP'), ('1973', 'CD'), ('to', 'TO'), ('be', 'VB'), ('referred', 'VBN'), ('to', 'TO'), ('as', 'IN'), ('the', 'DT'), ('Code', 'NNP'), ('to', 'TO'), ('quash', 'VB'), ('the', 'DT'), ('complaint', 'NN'), ('which', 'WDT'), ('is', 'VBZ'), ('registered', 'VBN'), ('as', 'IN'), ('Criminal', NNP'), ('Case', 'NNP'), ('No', 'NNP'), ('2793', 'CD'), ('of', IN'), ('1996', 'CD'), ('before', 'IN'), ('the', 'DT'), ('Metropolitan', 'NNP'), ('Magistrate', 'NNP'), ('Court', 'NNP'), ('No', 'NNP'), [('16', 'CD'), ('Ahmedabad', 'NNP'), ('for', 'IN'), ('the', 'DT'), ('offences', 'NNS'), ('punishable', 'JJ'), ('under', IN'), ('section', 'NN'), ('138', 'CD'), ('of', 'IN'), ('the', 'DT'), ('Negotiable', 'JJ'), ('Instruments', NNS'), ('Act', 'NNP'), ('1881', 'CD').

Extracted named entities are listed below.:

TABLE 1. ENTITY NAME & CATEGORIES

Named Entity	**Entity Name**
Person	Author: M Kadri
Location	Ahmedabad
Date	1973
Organization	Anchor Capitals Of India Ltd
Court	Metropolitan Magistrate Court
Legal	section 138
Act	Negotiable Instruments Act
Case_no	Case No. 2793

II. DATASET

The dataset used consists of detailed legal case information, typically spanning around 20-40 pages per document. Each case captures all essential elements, including the court where the case was heard, relevant legal sections, acts, and laws cited, as well as the individuals involved. Additionally, it covers the locations tied to the case proceedings. This comprehensive dataset allows for rich analysis, making it ideal for extracting named entities and exploring relationships between various legal components like courts, laws, and people within the legal framework.

III. RELATED WORK

Paper [4] comprehensively reviewed Named Entity Recognition, emphasizing the prevalence of three-class classification in existing research: organizations, persons, and places. However, the integration of a knowledge base in NER systems was lacking. To extend NER to other domains, the first step is to establish a relevant knowledge base. In [5] , the paper examined NER for the same three classes (Location, Person and Organization), using a supervised learning approach with K-nearest neighbor to achieve its goals. The research in [6] explored Semantic Bootstrapping for NER on large unlabeled datasets. The Stanford NER Tagger (2017) [7] expanded NER to recognize additional classes but at the cost of reduced accuracy, particularly when processing sentences in lowercase, where it struggled to identify entities. In the digital age, new approaches involving machine learning and neural network architectures have emerged to address NER challenges. Rule-based learning has been a cornerstone of research for many years. Rule Learning Systems (RLS) differ in their approaches to rule generation, rule prioritization, and entity extraction. Addressing the issues, we applied rule-based machine learning approach to identify legal-specific named entities such as *Date*, *Location*, *Organization*, *Person*, *Court*, *Legal Act*, and *Case Number*, thereby improving the accuracy and relevance of NER in the legal domain.

A. Similarity Measure for the Entities

To cluster extracted legal entities effectively, it's essential to identify features that determine their similarity. This approach uses WordNet to compute semantic similarity between legal entities by analyzing their relationships in a hierarchical structure. By assessing the path relationships between legal terms, such as court names, case references, or legal terms, a similarity score quantifies how closely related they are. This score aids in clustering entities like *Person*, *Organization*, *Court*, *Case Number*, *Legal Act*, and *Location*. Path-based and information content-based metrics enhance clustering accuracy in legal text.

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:03:29 UTC from IEEE Xplore. Restrictions apply.

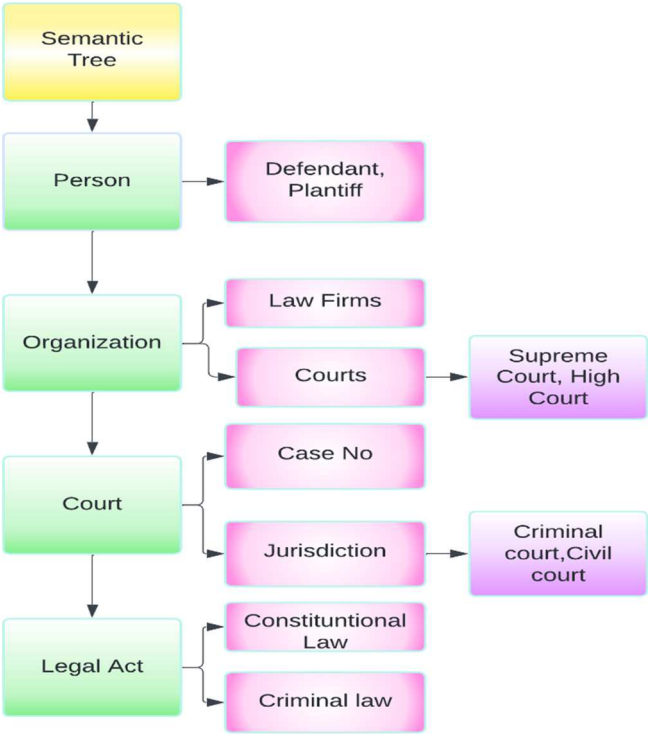

Fig. 1. Semantic Tree

B. Term Frequency-Inverse Document Frequency

Term Frequency-Inverse Document Frequency vectorization one of the method in Natural Language Processing (NLP) that encodes text data into numerical representations. It assesses the significance of words within a document compared to the entire collection of documents. Term Frequency measures the occurrence of a word within a document, while Inverse Document Frequency measures the word's rarity across the entire document collection. Mathematically, TF calculates the frequency of a word within a document relative to the total number of words in that document.

$$\mathrm{TF}(t,d) = \frac{\text{Number of times term t appears in document d}}{\text{Total number of terms in document d}}$$

IDF is calculated as:

$$\mathrm{IDF}(t,D) = \log\frac{N}{(\text{Number of documents containing term t})}$$

Where N is the total number of documents in the corpus. Final score is obtained by multiplying these both values:

$$\mathrm{TF-IDF}(t,d,D) = \mathrm{TF}(t,d) \times \mathrm{IDF}(t,D)$$

C. K-means clustering:

K-means clustering is one of the most popular used unsupervised learning algorithm partitioning data points to a predefined number of clusters, K, based on their similarity. It works by minimizing the variance within clusters and maximizing the variance between them. The algorithm begins by randomly initializing K centroids, which act as the centers of the clusters. Each data point is then assigned to the closest centroid based on a distance metric, typically Euclidean distance.

The process iterates through two main steps:

- Assignment step: In the assignment step, each data point is allocated to the cluster corresponding to the nearest centroid based on the chosen distance metric.
- Update step: In the update step, the centroids are updated by recalculating their positions as the mean of all data points assigned to each cluster.

These steps are repeated iteratively until the algorithm converges, which occurs when the centroids stabilize and no longer change, or the cluster assignments remain consistent. K-means is efficient for large datasets but may converge to a local optimum, which is why techniques like multiple initializations (k-means++) and the use of additional metrics like the silhouette scores which is often used to assess the quality of the clustering.

D. Silhouette Score:

The Silhouette Score evaluates how well an object fits within its assigned cluster compared to how well it fits within other clusters.. Range is -1 to 1. For each data point, the silhouette score is calculated using the formula:

$$s = \frac{b-a}{\max(a,b)}$$

E. KDTree

K-dimensional tree (KDTree) is a data structure designed to organize points in k-dimensional space, making it efficient for multidimensional data processing. It is commonly used in applications like nearest neighbor searches, range searches, and clustering tasks. The KDTree is built by dividing the space recursively along its axes, forming a binary tree where each node corresponds to a point in a k-dimensional space. This structure significantly reduces the search space compared to brute-force methods. In machine learning, KDTree enhances clustering algorithms like K-means by accelerating the search for nearest cluster centroids, leading to faster computation times and improved performance in high-dimensional datasets.

IV. PROPOSED FRAMEWORK

A. Text Preprocessing and Named Entity Recognition:

The process starts with sentence tokenization, where the input text is split into individual sentences using the sent_tokenize() function from spaCy. This step helps in isolating entities so that each sentence can be processed independently. Then, word tokenization is performed for every sentence using word_tokenize() from spaCy, splitting the sentence into words and removing some of the punctuation. Finally, every word is tagged with a Part-of-Speech tag that will tell what grammatical role the word plays (such as noun or verb). Named entity extraction involves the final stage where chunking takes place. It identifies a named entity by focusing attention on proper nouns and certain grammatical patterns. To enhance the identification of important entities in text, regular expressions is used to identify phrases of proper nouns, such as "Supreme Court.

B. Named Entity Similarity Calculation:

At this stage, semantic similarity is measured for the named entities extracted using the Term Frequency-Inverse

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:03:29 UTC from IEEE Xplore. Restrictions apply.

Document Frequency (TF-IDF) method. Every named entity will be tokenized and then categorized into grammatical classes using Part-of-Speech tagging. The tokens then will be vectorized using TF-IDF for the generation of numerical representations. This method compares the frequencies of words along with the importance of words within the whole dataset. The similarity between two entities is determined using the cosine similarity metric, which measures the cosine of the angle between the vectors representing the entities in the TF-IDF space. This yields a matrix of similarities where scores on all pairs of named entities are stored, ready to be used for clustering purposes. This approach avoids the traditional methods such as WordNet's Wu-Palmer Similarity metric and focuses instead on the more general, context-independent representation of words using TF-IDF.

C. Clustering of Named Entities:

In this step, named entities with similar characteristics are grouped into clusters based on their similarity scores from the similarity matrix.The K-Means clustering algorithm is applied to this matrix to organize the named entities into distinct clusters. Since the optimal number of clusters, K, is initially unknown, the algorithm tests several values for K, ranging from 2 up to a predetermined maximum. To evaluate the quality of clustering for various K values, the Silhouette Score is the primary metric used. This score evaluates the cohesiveness within clusters and their distinctiveness from one another, with higher values signifying more well-defined and separated clusters. The optimal K value is selected by maximizing the Silhouette Score, ensuring effective grouping of named entities. To enhance clustering efficiency, integrating KDTree can significantly reduce computational overhead and accelerate distance calculations, especially beneficial for large or high-dimensional datasets. This approach balances computational efficiency with cluster quality, making it ideal for practical applications in clustering tasks.

D. Cluster Labeling:

After the named entities are clustered, each cluster is assigned a label from predefined categories such as PERSON, GPE (Geopolitical Entity), ORG (Organization), DATE, ACT, COURT**,** and CASE_NO**:** Named Entity Recognition (NER) with spaCy: The entities within each cluster are analysed using spaCy's pre-trained NER model. This model is capable of recognizing and classifying entities such as people, locations, organizations, court, etc. Regular Expressions for Specific Legal Entities: Since the system is designed for legal cases (e.g., cheque bounce cases), specific regular expressions are used to identify legal terms.

For example:

Case numbers (using patterns like "Case No. 12345").Legal acts (e.g., "Negotiable Instruments Act").Courts (e.g., "Supreme Court", "High Court").

Dominant Label Assignment: Based on the type of entities identified within each cluster, the cluster is labeled with the dominant category. For example, if a cluster contains many person names, it will be labeled PERSON, or if it contains legal acts, it will be labeled ACT.

E. Final Output:

The final output comprises clustered named entities organized according to their semantic similarity. Each cluster is labeled with specific categories such as PERSON, GPE, DATE, ORG, ACT, COURT, CASE_NO, and others. This structured output enhances our understanding of the entities within the text, enabling categorization based on entity type. Furthermore, it enables deeper analysis of the relationships and patterns among the entities, providing valuable insights into the data.

Steps taken in the algorithm are explained below:

- *Entity Extraction:* The framework begins by loading a pre-trained spaCy model for processing legal text. It uses regex patterns to identify legal entities such as case numbers, acts, courts, and legal terms. The extract_entities_from_sentence function extracts named entities and custom legal terms, organizing them into a DataFrame categorized by entity types (e.g., Person, Location, Date).
- *Similarity Calculation:* After extraction, the framework calculates similarity scores among the entities using TF-IDF vectorization and cosine similarity. The calculate_similarity function generates a similarity score matrix to reveal relationships between the entities.
- *Clustering:* The framework employs K-means clustering to group similar entities, utilizing the KDTree structure to optimize centroid lookups. The fast_kmeans_with_kdtree function iteratively tests different values of k (number of clusters), selecting the optimal configuration based on silhouette scores proposed in the future work of paper [3].
- *Results Display:* The clustering results are consolidated and presented through the display_clusters_and_majority_label function. This function groups entities by their assigned clusters, highlighting the majority label within each cluster for clear interpretation of relationships among entities.

Algorithm 1: Extract Sentences

sentences ← *sentence extract (input_text)*
// Extract sentences from the input text

Algorithm 2: Initialize Data Structure

data ← *[] // Prepare a list to hold extracted data for DataFrame*

Algorithm 3: For Each Sentence

For i ← 1 to len(sentences) do
sentence_text ← normalize_text(sentences[i]) // Normalize the text
entities ← extract_entities_from_sentence(sentence_text) // Extract entities from the normalized sentence
For Each Entity Type:
For entity_type, entity_values in entities do

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:03:29 UTC from IEEE Xplore. Restrictions apply.

For Each Entity Value:
For value in entity_values do
data.append((entity_type,value)) // Append the entity type and value to the data list

Algorithm 4: Create DataFrame

entities_df ← pd.DataFrame(data, columns=["Entity Type", "Entity"]) // Create a DataFrame from the extracted data

Algorithm 5: Calculate Similarity Scores

vectors ← TfidfVectorizer().fit_transform(entities_df['Entity']).toarray() // Vectorize the entities using TF-IDF
1) cosine_sim ← cosine_similarity(vectors) // Calculate cosine similarity

Algorithm 6: Perform Clustering

1) best_k ← 2, best_clusters ← None, best_silhouette ← -1
2) distance_matrix ← 1 - cosine_sim
3) points ← 1 - cosine_sim
4) kdtree ← KDTree(points, metric="manhattan")
5)
6) // Clustering for k = 2 to max_k
7) For k ← 2 to max_k do
8) kmeans ← KMeans(n_clusters=k, random_state=0, n_init=10)
9) clusters ← kmeans.fit_predict(points)
10) kdtree ← KDTree(kmeans.cluster_centers_)
11)
12) silhouette_avg ← silhouette_score(points, clusters, metric="cosine")
13) If silhouette_avg > best_:
14) best_silhouette ← silhouette_avg,
15) best_k ← k, best_clusters ← clusters
16) // Output results
17) Print("Best K:", best_k, "Silhouette Score:", best_silhouette)
18) entities_df ← DataFrame(["Entity", "Cluster"], values=[all_entities, best_clusters])
19) // Display clusters
20) Function display_clusters(df):
21) For each cluster, group in df.groupby("Cluster"):
22) majority_label ← map_majority_label(group['Entity'].value_counts().idxmax())
23) Print("Cluster", cluster, "(Label:", majority_label, "):", join(group['Entity'], ', '))
24) Print("Clusters:")
display_clusters(entities_df)

Algorithm 7:Assign Clusters to DataFrame

entities_df['Cluster'] ← best_clusters // Assign the best clusters to the DataFrame

Algorithm 8: Display Clusters

grouped ← entities_df.groupby('Cluster')
For cluster, group in grouped do

entity_type_counts ← group['Entity Type'].value_counts() // Count entity types in the cluster

majority_label ← entity_type_counts.idxmax() // Determine majority label for the cluster

entities ← group['Entity'].tolist() // Get entities in the current cluster

Print Cluster {cluster} (Majority Label: {majority_label}):

Print - " + ', '.join(entities)

V. EVALUATION

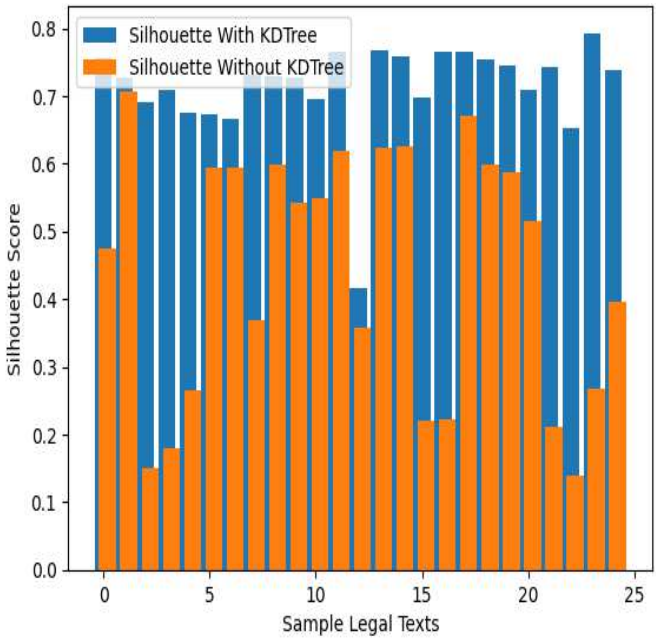

Fig. 2. Silhouette Score with and without KDTree

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:03:29 UTC from IEEE Xplore. Restrictions apply.

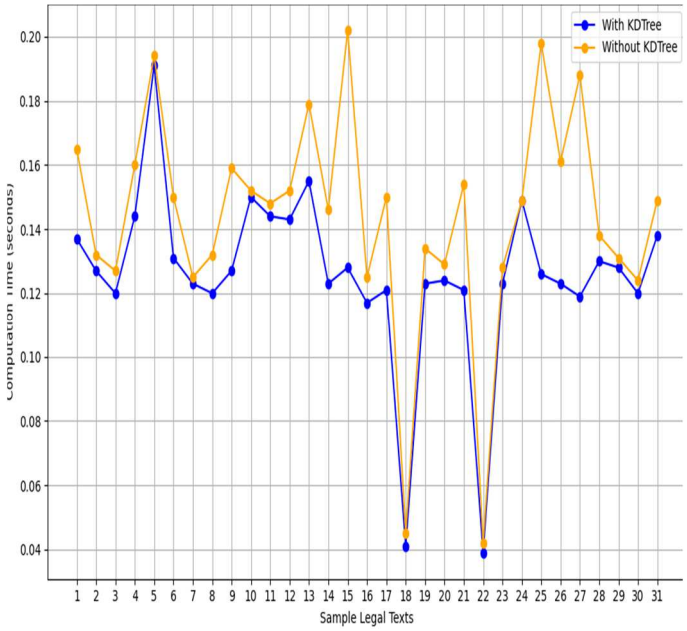

Fig. 3. Computation Time Comparison with and without KDTree

The integration of KDTree into K-Means clustering demonstrates significant improvements in both cluster quality and computational efficiency, particularly when applied to legal text datasets. The first graph highlights the Silhouette Scores achieved with and without KDTree. The results clearly show that using KDTree leads to high Silhouette Scores, which indicating better clusters. Not only are the scores consistently higher, but they also exhibit reduced variability across different trials, reflecting a more stable and reliable clustering performance. In some cases, the improvement is particularly significant, showcasing the advantages of KDTree for handling large and complex legal datasets.

The second graph provides further evidence of KDTree's benefits, demonstrating a noticeable reduction in computation time when compared to clustering without KDTree. By accelerating distance calculations, KDTree addresses scalability challenges, making K-Means more efficient for resource-intensive tasks. This reduction in computational overhead is particularly important when working with extensive legal documents, where processing speed and scalability are critical.

From a practical perspective, these improvements are highly relevant for legal text processing tasks such as case triaging, document clustering, information retrieval, and organization of complex legal records. KDTree effectively manages the linguistic and semantic complexities inherent in legal text embeddings, enabling more accurate and efficient clustering. In conclusion, KDTree integration not only enhances the quality of clusters but also ensures scalability and resource efficiency, making it an essential tool for organizing and analyzing complex legal information.

VI. CONCLUSION AND FUTURE WORK

Very useful is using KDTree for K-means clustering as seen in the graph and analysis; it greatly reduces the computation time while still keeping high quality in clustering, which is very important when processing large datasets or applications with a time constraint. The combination of KDTree along with the metrics such as Silhouette Score has made it possible for quicker extraction and clustering of named entities. Further improvement may be done in legal text analysis and clustering systems based on future research by developing methods for refinement of techniques in entity extraction, exploring sophisticated similarity measures, experimenting with different types of clustering algorithms, and a larger set of evaluation metrics for clusters. All of these are highly enriching the field of legal text analysis and provide worthwhile resources for legal practitioners: developing interactive visualizations, integrating the system into legal reasoning. Frameworks also enhance scalability and efficiency, and can be tailored for specific legal domains.

REFERENCES

[1] P. Patil, A. Ranade, M. Sabane, O. Litake, and R. Joshi, “L3Cube-MahaNER: A Marathi Named Entity Recognition Dataset and BERT models.” [Online]. Available: https://github.com/l3cube-pune/MarathiNLP

[2] M. B. Shelke, D. D. Sawant, C. B. Kadam, K. Ambhure, and S. N. Deshmukh, “Marathi SentiWordNet: A lexical resource for sentiment analysis of Marathi,” Concurrency and Computation: Practice and Experience, vol. 35, no. 2, Jan. 2023, doi: 10.1002/cpe.7497.

[3] S. P. Tripathi and H. Rai, "SimNER – An Accurate and Faster Algorithm for Named Entity Recognition," 2018 Second International Conference on Advances in Computing, Control and Communication Technology (IAC3T), Mumbai, India, 2018, pp. 1-6, doi: 10.1109/IAC3T.2018.8659025..

[4] M. Marrero, J. Urbano, S. Sánchez-Cuadrado, J. Morato, and J. M. Gómez-Berbís, "Named Entity Recognition: Fallacies, challenges and opportunities," *Computer Standards & Interfaces*, vol. 35, no. 5, pp. 482-489, 2013, doi: 10.1016/j.csi.2012.09.004.

[5] X. Liu, S. Zhang, F. Wei, and M. Zhou, "Recognizing Named Entities in Tweets," Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, Portland, OR, USA, 2011, pp. 359-367.

[6] A. Ritter, S. Clark, Mausam, and O. Etzioni, "Named Entity Recognition in Tweets: An Experimental Study," Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing, Edinburgh, Scotland, UK, 2011, pp. 1524-1534.

[7] C. Manning, M. Surdeanu, J. Bauer, J. Finkel, S. Bethard, and D. McClosky, "The Stanford CoreNLP Natural Language Processing Toolkit," in Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations, Baltimore, Maryland, 2014, pp. 55-60. doi: 10.3115/v1/P14-5010.

[8] N. Patil, A. Patil, and B. V. Pawar, "Hybrid Approach for Marathi Named Entity Recognition," in Proceedings of the 14th International Conference on Natural Language Processing (ICON-2017), Kolkata, India, Dec. 2017, pp. 103-111.

[9] X. Bai and L. J. Latecki, “Path similarity skeleton graph matching,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 30, no. 7, pp. 1282–1292, Jul. 2008, doi: 10.1109/TPAMI.2007.70769.

[10] Z. Wu and M. Palmer, "Verb Semantics and Lexical Selection," *arXiv preprint*, arXiv:cmp-lg/9406033, 1994.

[11] M. Marrero, J. Urbano, S. Sánchez-Cuadrado, J. Morato, and J. M. Gómez-Berbís, "Named Entity Recognition: Fallacies, challenges and opportunities," *Computer Standards & Interfaces*, vol. 35, no. 5, pp. 482-489, 2013. doi: 10.1016/j.csi.2012.09.004.

Authorized licensed use limited to: Ramarao Adik Institute of Technology. Downloaded on February 09,2026 at 11:03:29 UTC from IEEE Xplore. Restrictions apply.